SOFTWARE REQUIREMENTS SPECIFICATIONS

Air Ticket Reservation System

Project Code:	ATRS-2025-SRS
Internal Advisor:	
External Advisor:	
Project Manager:	Muhammad Ehtisham
Project Team:	Muhammad Ehtisham, Malik Muhammad Umair, Ali Zain
Submission Date:	

Project Manager's Signature

Document Information

Customer	
Project	Air Ticket Reservation System
Document	Software Requirements Specifications
Document Version	4.0
Identifier	ATRS-2025-SRS
Status	Final Draft
Author(s)	Muhammad Ehtisham, Malik Muhammad Umair, Ali Zain
Approver(s)	Project Manager / Internal Advisor
Issue Date	
Document Location	Project Repository / Documentation Folder
Distribution	1. Internal Advisor 2. Project Manager 3. Project Team

Definition of Terms, Acronyms and Abbreviations

This section provides definitions of all terms, acronyms, and abbreviations required to correctly interpret this document.

Term / Acronym	Definition
ATRS	Air Ticket Reservation System
SRS	Software Requirements Specifications
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
CNIC	Computerized National Identity Card
GUI	Graphical User Interface (N/A – console-based system)
CLI	Command Line Interface – the text-based user interface used by ATRS
DIA	Dubai International Airport
SIA	Sharjah International Airport
AIA	Abu Dhabi International Airport (UAE) / King Abdulaziz International Airport (Jeddah, SA)
KIA	King Khalid International Airport, Riyadh
FIA	King Fahd International Airport, Dammam
LHE	IATA code for Allama Iqbal International Airport, Lahore
ISB	IATA code for New Islamabad International Airport
FSD	IATA code for Faisalabad International Airport
PIA	Pakistan International Airlines
PKR	Pakistani Rupee – the base currency for pricing in ATRS
AED	UAE Dirham
SAR	Saudi Riyal
GMT	Greenwich Mean Time
OOP	Object-Oriented Programming
STL	Standard Template Library (C++)
RS	Requirements Specifications
PM	Project Manager

Table of Contents

1. Introduction
  1.1 Purpose of Document
  1.2 Project Overview
  1.3 Scope
2. Overall System Description
  2.1 User Characteristics
  2.2 Operating Environment
  2.3 System Constraints
3. External Interface Requirements
  3.1 Hardware Interfaces
  3.2 Software Interfaces
  3.3 Communications Interfaces
4. Functional Requirements
  4.1 User Authentication Module
    4.1.1 FR-UA-01: User Sign-Up
    4.1.2 FR-UA-02: User Login
    4.1.3 FR-UA-03: View Account Details
    4.1.4 FR-UA-04: Edit Account Details
    4.1.5 FR-UA-05: Forgot Password / Password Recovery
  4.2 Flight Search Module
    4.2.1 FR-FS-01: Search Available Flights
    4.2.2 FR-FS-02: Flight Information Display
  4.3 Booking Process Module
    4.3.1 FR-BP-01: Passenger Details Entry
    4.3.2 FR-BP-02: Flight Class Selection
    4.3.3 FR-BP-03: Seat Selection
    4.3.4 FR-BP-04: Baggage Weight and Fare Calculation
    4.3.5 FR-BP-05: Booking Confirmation
  4.4 Payment Module
    4.4.1 FR-PM-01: Currency Selection and Conversion
    4.4.2 FR-PM-02: Payment Methods
    4.4.3 FR-PM-03: Payment Validation and Confirmation
  4.5 Receipt Module
    4.5.1 FR-RC-01: Receipt Generation
  4.6 Flight Cancellation Module
    4.6.1 FR-CF-01: Cancel Flight
5. Non-Functional Requirements
  5.1 Performance Requirements
  5.2 Safety Requirements
  5.3 Security Requirements
  5.4 Reliability Requirements
  5.5 Maintainability Requirements
  5.6 Usability Requirements
  5.7 Scalability Requirements
  5.8 Availability Requirements
  5.9 Backup and Recovery Requirements
  5.10 User Documentation
6. Assumptions and Dependencies
7. References
8. Appendices

1. Introduction

1.1 Purpose of Document

This Software Requirements Specifications (SRS) document describes the complete and detailed functional and non-functional requirements of the Air Ticket Reservation System (ATRS). It serves as the authoritative reference for the development team, the QA team, the internal advisor, and all other project stakeholders throughout the system development lifecycle.

This document defines what the system must do, not how it does it. It ensures that all parties share a common, unambiguous understanding of the system's expected behaviour. The SRS is directly derived from and validated against the actual C++ implementation to guarantee accuracy and traceability.

Intended audience: Project Manager, Developers, QA Engineers, Internal Advisor, External Advisor.

1.2 Project Overview

The Air Ticket Reservation System (ATRS) is a console-based, object-oriented C++ application designed to enable passengers to search, book, and manage domestic-to-international air ticket reservations. The system supports the entire reservation lifecycle — from account creation and flight discovery through to seat selection, payment, receipt generation, and cancellation.

The system focuses on flights departing from three major Pakistani airports — Allama Iqbal International Airport (Lahore), Faisalabad International Airport, and New Islamabad International Airport — to destinations in two countries: the United Arab Emirates (UAE) and the Kingdom of Saudi Arabia (KSA).

The system is implemented using an OOP class hierarchy: Person → User_authentication → Booking → Payment → Receipt, with the Flights, Search_flight, and Seat_management classes providing supporting services. All data is maintained in volatile in-memory storage using C++ STL containers (vector, map, array).

Key benefits of the system include: streamlined reservation workflow, multi-passenger booking support, multi-currency payment processing, and real-time seat availability tracking within a session.

1.3 Scope

The following capabilities ARE within the scope of the ATRS:

- User account management: registration, login, profile viewing, profile editing, and password recovery.
- Flight search: querying available flights by departure airport, arrival country, arrival airport, travel date, and trip type (one-way or return).
- Flight information display: showing complete flight details including flight ID, airline name, route, dates, pricing, baggage allowance, timings, and refund/upgrade policy.
- Booking process: entering passenger details (up to 5 per booking), selecting flight class, choosing available seats, specifying baggage weight, and computing total fare.
- Payment processing: supporting debit card, credit card, and online banking payment methods; supporting 5 currencies (PKR, USD, EUR, GBP, AED, SAR) with automatic conversion.
- Receipt generation: displaying a complete booking confirmation with all passenger and flight details.
- Flight cancellation: allowing users to cancel their booked flight using their CNIC, which releases the associated seat.
- Input validation: graceful handling of all invalid, out-of-range, and non-numeric inputs throughout the system.

The following capabilities are OUT of scope for this version:

- Persistent data storage — all data is in-memory only and is lost when the program terminates.
- Graphical user interface — the system is entirely console-based.
- Real-time integration with airline APIs or external flight databases.
- Actual OTP/SMS delivery for password recovery (the OTP step is simulated).
- Real payment gateway integration — card and bank details are collected but not validated against real payment systems.
- Concurrent multi-user access — the system is single-process, single-user.
- Email notifications, booking reminders, or any network-based communication.
- International flights other than UAE and Saudi Arabia.

2. Overall System Description

This section describes the environment in which the ATRS will be developed and used, the anticipated users of the system, and the known constraints, assumptions, and dependencies.

2.1 User Characteristics

The ATRS serves a single user class interacting through a command-line interface. The following characteristics apply:

Attribute	Description
User Type	General public — any person wishing to book an air ticket
Technical Proficiency	Basic — capable of reading console prompts and entering text/numeric input
Language	English — all prompts and messages are in English
Authentication	All booking-related functions require a registered and authenticated account
Concurrent Users	Single user per program instance (no multi-user concurrency)
Special Categories	None — no admin or superuser role is implemented in this version

2.2 Operating Environment

The ATRS is designed to operate in the following environment:

Component	Specification
Application Type	Console-based command-line application
Programming Language	C++ (Standard: ISO C++11/14/17)
Compiler	GNU g++ (GCC 9.0 or higher); also compatible with MSVC and Clang
Operating System	Windows 10/11, Ubuntu Linux 20.04+, macOS 10.15+ (any OS with g++ support)
Hardware – Minimum	1 GHz CPU, 512 MB RAM, 10 MB free disk space for binary and data
Standard Libraries	C++ STL: <iostream>, <string>, <vector>, <map>, <array>, <algorithm>, <chrono>, <ctime>
Execution Model	Single-process, single-threaded, interactive console session
Data Storage	Volatile in-memory storage using STL containers (no file I/O or database)
Network	None required — standalone offline application

2.3 System Constraints

The following constraints govern the design and operation of the ATRS:

Software Constraints:

- The system is implemented exclusively in C++ and does not use any external frameworks, databases, or third-party libraries.
- All data is stored in volatile in-memory STL containers; no file I/O is performed. All data is lost upon program termination.
- The date validation logic restricts valid flight dates to calendar years 2025 and 2026 only (hardcoded constraint in the source code).
- The system supports exactly 3 departure airports (Lahore, Faisalabad, Islamabad) and 2 destination countries (UAE, Saudi Arabia) with 3 airports each — a total of 36 unique flight IDs (18 one-way + 18 return).
- The maximum number of passengers per booking is hard-limited to 5.
- Each flight has exactly 50 seats: Economy (1–30), Business (31–40), First Class (41–50).
- Base prices are compile-time constants: UAE flights = PKR 100,000; Saudi Arabia flights = PKR 150,000.

Hardware Constraints:

- No specialized hardware is required. The system runs on any standard desktop or laptop computer.

Cultural and Language Constraints:

- All user-facing text is in English only. No multi-language support is provided.
- Date input format is dd/mm/yyyy. All times are expressed in GMT offsets.

Legal Constraints:

- No real payment processing is performed; no PCI-DSS compliance is required for this version.
- No personally identifiable information (PII) is persisted; all user data is in-memory only.

User Constraints:

- The user must interact via a text-based console. No graphical controls are available.
- The user must follow exact input format requirements (e.g., date as dd/mm/yyyy, airport codes as DIA/SIA/AIA/KIA/FIA).

3. External Interface Requirements

This section specifies all requirements that ensure the ATRS connects correctly to its external environment, including hardware, software, and communication interfaces.

3.1 Hardware Interfaces

The ATRS does not interface directly with any specialized hardware components. All interaction is mediated through the host operating system's standard input/output streams. The following hardware is implicitly required:

- Standard keyboard for text input via stdin (std::cin).
- Standard display monitor for console output via stdout (std::cout).
- Sufficient RAM to hold in-memory data structures (vector, map) — estimated at under 10 MB for typical session data.
- System clock access — the system reads the local system clock via std::chrono and std::localtime to validate flight dates against the current date.

No printers, network cards, card readers, or other peripheral hardware interfaces are required or supported.

3.2 Software Interfaces

The ATRS is a self-contained C++ application and does not interface with any external software systems, databases, or middleware in the current version. The following internal software interfaces exist:

Interface	Description
C++ Standard Library (STL)	The system uses STL containers (std::vector, std::map, std::array) for all in-memory data storage, and STL algorithms (std::find) for searching.
<chrono> / <ctime>	Used by check_valid_date() to retrieve the current system date and compare against user-entered flight dates.
Operating System – stdin/stdout	All user interaction is conducted through the OS standard I/O streams; no GUI toolkit or window manager is used.
Compiler Toolchain	The application must be compiled using a C++11/14/17-compliant compiler (g++, Clang, MSVC). No runtime libraries beyond the C++ standard library are required.

3.3 Communications Interfaces

The current version of the ATRS does not use any network communication protocols, APIs, email services, or web services. All operations are performed locally on a single machine.

Note: The forgot password feature includes a simulated OTP flow (the user is prompted to enter a 4-digit code that was supposedly sent to their phone). In this version, any 4-digit string is accepted. A future version may integrate an SMS gateway or email service for actual OTP delivery.

4. Functional Requirements

This section defines all functional requirements of the ATRS, organised by module. Each requirement is assigned a unique identifier in the format FR-[MODULE]-[NN] and is directly traceable to the implemented source code.

4.1 User Authentication Module

The User Authentication Module manages all aspects of user identity: account creation, login, profile management, and password recovery. It is implemented in the User_authentication class, which inherits from the Person base class.

4.1.1 FR-UA-01: User Sign-Up

Feature ID	Feature Name	Priority	Module
FR-UA-01	User Sign-Up	High	User Sign-Up

Allows a new user to create an account by providing personal and security credentials.

Description	A new user registers by entering personal details, an email address, and a password that satisfies the defined security criteria.
Trigger	User selects option 1 (SignUp) from the main menu.
Inputs	First name (alphabetic only), Last name (alphabetic only), CNIC (numeric), Passport number (numeric), Phone number (numeric), Email address (string), Password (string)
Processing	1. System prompts for first and last name; validates using check_valid_name() — rejects digits and the following special characters: ` ~ ! @ # $ % ^ & *) (- _ > < , [ ] { } " ? : ; 2. System prompts for CNIC, passport, and phone number as integers; uses cin error-handling to reject non-numeric input. 3. System prompts for email (no format validation beyond string entry). 4. System enforces password criteria: minimum 8 characters; must contain at least one uppercase letter, one lowercase letter, one digit, and one punctuation character. 5. On success, stores the User_authentication object in the static user vector and sets check_signup = true.
Outputs	Success: 'Account Created successfully!' Failure (name): 'Numbers and special characters are not allowed!' Failure (password length): 'Password must have atleast 8 characters long!' Failure (password criteria): 'Password does not match with criteria Try Again!'
Pre-Conditions	The system is running. No prior account is required.
Post-Conditions	A new User_authentication object is appended to the static user vector and is available for login within the current session.

Individual Requirements:

FR-UA-01.1	The system shall allow a user to register by entering first name, last name, CNIC, passport number, phone number, email address, and password.
FR-UA-01.2	The system shall reject any name (first or last) that contains numeric digits or any of the following special characters: ` ~ ! @ # $ % ^ & *) (- _ > < , [ ] { } " ? : ;
FR-UA-01.3	The system shall re-prompt for name input upon validation failure without terminating the registration flow.
FR-UA-01.4	The system shall enforce password complexity: minimum 8 characters with at least one uppercase letter, one lowercase letter, one digit, and one special/punctuation character.
FR-UA-01.5	The system shall re-prompt for password input upon validation failure without terminating the registration flow.

FR-UA-01.6	The system shall handle non-numeric input gracefully for CNIC, passport, and phone number fields by clearing the input buffer and re-prompting.
FR-UA-01.7	Upon successful registration, the system shall display 'Account Created successfully!' and persist the user record in the in-memory user store.

4.1.2 FR-UA-02: User Login

Feature ID	Feature Name	Priority	Module
FR-UA-02	User Login	High	User Login

Allows a registered user to authenticate using email and password.

Description	A registered user authenticates by providing their email address and password. The system searches the in-memory user store for a matching record.
Trigger	User selects option 2 (Login) from the main menu, or selects Login within the booking flow.
Inputs	Email address (string), Password (string)
Processing	System iterates the static user vector comparing the entered email and password against stored credentials. Returns 0 on match (success) or -1 on failure.
Outputs	Success: 'Login successful' Failure: 'Email or password does not match with user credentials!'
Pre-Conditions	At least one user account must exist in the in-memory user store.
Post-Conditions	On success, the booking flow may proceed. No persistent session token is created.

Individual Requirements:

FR-UA-02.1	The system shall authenticate a user by matching the entered email and password against all records in the in-memory user store (case-sensitive comparison).
FR-UA-02.2	On successful authentication, the system shall display 'Login successful' and return a success indicator (0).
FR-UA-02.3	On authentication failure, the system shall display 'Email or password does not match with user credentials!' and return a failure indicator (-1).
FR-UA-02.4	The system shall not limit the number of login attempts within a session.

4.1.3 FR-UA-03: View Account Details

Feature ID	Feature Name	Priority	Module
FR-UA-03	View Account Details	Medium	View Account Details

Allows an authenticated user to view their stored account details.

Description	A user verifies their identity using email and password, and the system displays their complete profile information.
Trigger	User selects option 3 (Check Account Details) from the main menu.
Inputs	Email address (string), Password (string)
Processing	System locates the matching user record and displays all stored fields.
Outputs	Success: Displays first name, last name, CNIC, phone number, email, and password. Failure: 'Email or password does not match with user credentials!'

Pre-Conditions	User account must exist.
Post-Conditions	No data modification occurs.

Individual Requirements:

FR-UA-03.1	The system shall require email and password verification before displaying account details.
FR-UA-03.2	The system shall display: first name, last name, CNIC, phone number, email address, and password for the authenticated user.
FR-UA-03.3	The system shall display an error message if the credentials do not match any stored record.

4.1.4 FR-UA-04: Edit Account Details

Feature ID	Feature Name	Priority	Module
FR-UA-04	Edit Account Details	Medium	Edit Account Details

Allows an authenticated user to update their profile information.

Description	A user verifies their identity and then updates their first name, last name, phone number, and password. CNIC, passport, and email are not editable in this version.
Trigger	User selects option 4 (Edit Account Details) from the main menu.
Inputs	Current email and password (for authentication), New first name, New last name, New phone number, New password
Processing	1. Authenticate user using email/password. 2. Apply same name validation (check_valid_name) to new names. 3. Accept new phone number with numeric validation. 4. Enforce same password criteria as sign-up. 5. Update the user record in-place in the static user vector.
Outputs	Success: 'Details changed successfully!' Failure: 'Email or password does not match with user credentials!'
Pre-Conditions	User account must exist; user must know their current credentials.
Post-Conditions	The user record in the in-memory store reflects the updated values.

Individual Requirements:

FR-UA-04.1	The system shall require email and password verification before allowing profile edits.
FR-UA-04.2	The system shall allow updates to: first name, last name, phone number, and password.
FR-UA-04.3	The system shall apply the same name validation rules (no digits or listed special characters) to the updated name fields.
FR-UA-04.4	The system shall enforce the same password complexity requirements for the new password.
FR-UA-04.5	The system shall not allow editing of CNIC, passport number, or email address.
FR-UA-04.6	Upon successful update, the system shall display 'Details changed successfully!'

4.1.5 FR-UA-05: Forgot Password / Password Recovery

Feature ID	Feature Name	Priority	Module
FR-UA-05	Forgot Password / Password Recovery	Medium	Forgot Password / Password Recovery

Allows a user to reset their password by verifying email and phone number, then entering a simulated OTP.

Description	A user who has forgotten their password can recover it by verifying their registered email and phone number combination. A simulated 4-digit OTP is required, after which a new password can be set.
Trigger	User selects option 5 (Forgot Password) from the main menu.
Inputs	Registered email address, Registered phone number, 4-digit OTP code, New password
Processing	1. Prompt for email and phone number; search the user store for a matching email+phone combination. 2. If no match: display error and return (not re-prompting — the function returns immediately). 3. If match found: prompt for a 4-digit OTP code (any 4-character string is accepted in this version). 4. Prompt for and validate new password against the same password criteria. 5. Update the password in the matching user record.
Outputs	Match found + valid OTP + valid password: 'Password Changed Successfully!' No match: 'Phone number or email does not exist in Database!' OTP wrong length: 'Password must have 4 digits!'
Pre-Conditions	The user must have a registered account with a known email and phone number.
Post-Conditions	The user's password in the in-memory store is updated to the new value.

Individual Requirements:

FR-UA-05.1	The system shall verify the combination of email address AND phone number before initiating password recovery.
FR-UA-05.2	If the email and phone combination does not match any record, the system shall display 'Phone number or email does not exist in Database!' and terminate the recovery flow.
FR-UA-05.3	The system shall accept any 4-character string as the OTP code (simulated delivery in this version).
FR-UA-05.4	The system shall enforce the standard password complexity rules for the new password.
FR-UA-05.5	Upon successful password reset, the system shall display 'Password Changed Successfully!' and update the record in-memory.

4.2 Flight Search Module

The Flight Search Module enables users to search for available flights by specifying a departure airport, destination country and airport, trip type, and travel date. It is implemented by the Search_flight class (which inherits from Flights) and the check_valid_date() function.

4.2.1 FR-FS-01: Search Available Flights

Feature ID	Feature Name	Priority	Module
FR-FS-01	Search Available Flights	High	Search Available Flights

Allows a user to search for and retrieve flight details by specifying all required search parameters.

Description	The user selects a departure airport, enters a destination country and airport, selects the trip type (one-way or return), and enters a departure date. The system returns the matching flight details.
Trigger	User selects option 6 (Search Flight) or option 7 (Flight Booking) from the main menu.
Inputs	Departure airport (1=Lahore, 2=Faisalabad, 3=Islamabad), Arrival country ('UAE' or 'Saudi Arabia'), Arrival airport code (DIA/SIA/AIA/KIA/FIA or 1/2/3), Flight type (1=One-Way, 2=Return), Departure date (dd/mm/yyyy)
Processing	1. Display available Pakistani departure airports. 2. Accept and validate departure airport selection (1, 2, or 3 only). 3. Accept and validate arrival country ('uae' or 'saudi arabia' — case-insensitive). 4. Display corresponding destination airports. 5. Accept and validate arrival airport code (DIA/SIA/AIA/KIA/FIA or equivalent numbers 1/2/3). 6. Accept and validate flight type selection (1 or 2). 7. Accept and validate departure date using check_valid_date(): must be within 2025–2026 range, a valid calendar date, and on or after today's date. 8. Invoke the appropriate Flights method (Allama_Iqbal, faisalabad, or islamabad) to retrieve and display flight details and return the flight ID.
Outputs	Matching flight details including: flight ID, flight name, type, from, to, departure date, price (PKR), baggage allowance tiers, departure time, arrival time, refund and upgrade availability. For return flights: also displays return date and return time. Invalid country/airport: 'You Entered Wrong Details!' or 'Please Enter Valid input!' Invalid date: 'Please input valid Date!' Invalid menu choice: 'Enter valid Input!'
Pre-Conditions	System is running. No authentication required for search.
Post-Conditions	Flight ID is returned for use in the subsequent booking and payment flow.

Departure Airports (Pakistan):

Code	Airport Name	City	Input Values
1	Allama Iqbal International Airport	Lahore	1
2	Faisalabad International Airport	Faisalabad	2
3	New Islamabad International Airport	Islamabad	3

Destination Countries and Airports:

Country	Airport Name	Code	Input Values
UAE	Dubai International Airport	DIA	DIA or 1
UAE	Sharjah International Airport	SIA	SIA or 2
UAE	Abu Dhabi International Airport	AIA	AIA or 3

Saudi Arabia	King Khalid Int'l Airport (Riyadh)	KIA	KIA or 1
Saudi Arabia	King Fahd Int'l Airport (Dammam)	FIA	FIA or 2
Saudi Arabia	King Abdulaziz Int'l Airport (Jeddah)	AIA	AIA or 3

Individual Requirements:

FR-FS-01.1	The system shall present the list of 3 Pakistani departure airports and accept only input values 1, 2, or 3.
FR-FS-01.2	The system shall accept arrival country input in a case-insensitive manner and support 'UAE' and 'Saudi Arabia' as the only valid destination countries.
FR-FS-01.3	The system shall accept arrival airport codes case-insensitively as text codes (DIA, SIA, AIA, KIA, FIA) or as numeric selections (1, 2, 3).
FR-FS-01.4	The system shall support two trip types: One-Way (1) and Return (2).
FR-FS-01.5	The system shall validate the departure date using the following rules: the year must be 2025 or 2026; the month must be between 1 and 12; the day must be between 1 and 31; the date must not be in the past relative to the current system date.
FR-FS-01.6	The system shall display a unique flight ID and complete flight information for the matching route and date combination.
FR-FS-01.7	For return flights, the system shall additionally display the return date (departure date + a flight-specific offset of 1, 2, or 3 days) and the return time.
FR-FS-01.8	The system shall display baggage allowance tiers: 10 KG at PKR 10,000 and 20 KG at PKR 20,000 per passenger.
FR-FS-01.9	UAE flights shall have a base price of PKR 100,000 (defined as UAE_PRICE). Saudi Arabia flights shall have a base price of PKR 150,000 (defined as SA_PRICE).
FR-FS-01.10	All departure times shall be 08:30:00 GMT+5. Arrival times shall be 10:00:00 GMT+5. Return times shall be 11:30 GMT+4 (UAE) or GMT+3 (Saudi Arabia).
FR-FS-01.11	All flights shall display 'Refund : Available' and 'Upgrade : Available'.
FR-FS-01.12	The system shall handle invalid inputs at each step by re-prompting without crashing.

4.2.2 FR-FS-02: Flight Information Display

Feature ID	Feature Name	Priority	Module
FR-FS-02	Flight Information Display	High	Flight Information Display

The system shall display comprehensive flight information for any searched flight.

Individual Requirements:

FR-FS-02.1	The system shall display the following fields for every flight: Flight ID, Flight Name, Type (One-Way/Return), From (departure city), To (destination), Departure Date, Price (PKR), Baggage Allowance tiers, Departure Time, Arrival Time, Refund policy, Upgrade availability.
FR-FS-02.2	Return flights shall additionally display: Return Date, Return Time.
FR-FS-02.3	The flight display output shall be generated both during the initial search and again as part of the receipt generation, ensuring consistency.

4.3 Booking Process Module

The Booking Process Module manages the complete pre-payment booking flow: user authentication within the booking context, passenger details entry, class selection, seat selection, baggage specification, and total fare computation. It is implemented in the Booking class, which uses multiple inheritance from User_authentication and Seat_management.

4.3.1 FR-BP-01: Passenger Details Entry

Feature ID	Feature Name	Priority	Module
FR-BP-01	Passenger Details Entry	High	Passenger Details Entry

Captures the details of all passengers being booked on the selected flight.

Description	After a user decides to book a flight, the system prompts for the total number of passengers and the breakdown by type (Adult, Child, Infant). For each passenger, full personal details are collected.
Trigger	User confirms booking intent ('Yes' to the booking confirmation prompt).
Inputs	Number of passengers (integer, 1–5), Number of adults (integer), Number of children (integer), Number of infants (integer), Per passenger: First name, Last name, CNIC, Passport, Phone number
Processing	1. Prompt for total passenger count; validate range 1–5. 2. Prompt for adult count (must not exceed total); if adults fill total, children and infant prompts are skipped. 3. If adults present, prompt for children; validate sum does not exceed total. 4. If remaining slots exist, prompt for infants; validate. 5. Rules: children and infants cannot travel without at least 1 adult. 6. For each passenger (loop), call Person::person_input() to collect and validate personal details.
Outputs	Prompts for each passenger detail; success leads to next booking step. Invalid count: 'PLease give valid input!' Child without adult: 'Child cant travel without An Adult!' Infant without adult: 'Infants cant travel without An Adult!' Exceeded limit: 'Maximum Limit Reached!'
Pre-Conditions	User is authenticated (login or sign-up completed within booking flow).
Post-Conditions	All passenger Person objects are stored in the person_vector.

Individual Requirements:

FR-BP-01.1	The system shall allow a maximum of 5 passengers per booking transaction.
FR-BP-01.2	The system shall accept passenger types: Adult, Child, and Infant.
FR-BP-01.3	The system shall enforce the constraint that children cannot travel without at least one accompanying adult.
FR-BP-01.4	The system shall enforce the constraint that infants cannot travel without at least one accompanying adult.
FR-BP-01.5	The sum of adults, children, and infants must equal the specified total passenger count.
FR-BP-01.6	For each passenger, the system shall collect: first name, last name, CNIC, passport number, and phone number, applying the same validation rules as user sign-up.

4.3.2 FR-BP-02: Flight Class Selection

Feature ID	Feature Name	Priority	Module
FR-BP-02	Flight Class Selection	High	Flight Class Selection

Allows each passenger to independently select a travel class.

Description	For each passenger in the booking, the user selects from Economy, Business, or First Class. Each class corresponds to a different range of seat numbers on the aircraft.
Trigger	After each passenger's personal details are entered.
Inputs	Class choice: 1 (Economy), 2 (Business), 3 (First Class)
Processing	Displays the available seats for the chosen class, then prompts for seat selection.
Outputs	Lists available seats for the selected class. Invalid choice: 'Please enter a Valid number!'
Pre-Conditions	Passenger details have been entered.
Post-Conditions	System proceeds to seat selection for the chosen class.

Individual Requirements:

FR-BP-02.1	The system shall support three travel classes: Economy (seats 1–30), Business (seats 31–40), and First Class (seats 41–50).
FR-BP-02.2	Each passenger within the same booking may independently select a different travel class.
FR-BP-02.3	The system shall display only the available (unbooked) seats for the selected class.

4.3.3 FR-BP-03: Seat Selection

Feature ID	Feature Name	Priority	Module
FR-BP-03	Seat Selection	High	Seat Selection

Allows each passenger to select a specific available seat within their chosen class.

Description	The system displays available seat numbers within the selected class and the user selects one. The seat is immediately marked as booked in the in-memory seat map to prevent double-booking.
Trigger	After class selection for each passenger.
Inputs	Seat number (integer, within class range)
Processing	1. The seat map is keyed by flight ID and date string. 2. If the key combination does not yet exist, all 50 seats are initialised to 0 (available). 3. Validates that the selected seat number falls within the class range. 4. Validates that seats[id][date][seat-1] == 0 (not already booked). 5. Sets seats[id][date][seat-1] = 1 (booked). 6. Records the seat number in the booking object.
Outputs	Success: 'Seat selected successfully!' Already booked: 'Seat already Booked!' Wrong class range: 'Invalid seat number for this class!'
Pre-Conditions	Class has been selected; flight ID and date are known.
Post-Conditions	Seat is marked as booked in the static seat map (persists for the duration of the session).

Individual Requirements:

FR-BP-03.1	The system shall maintain a seat availability map keyed by flight ID and date string.
FR-BP-03.2	The system shall prevent double-booking: a seat that has been booked shall not be available for selection again until it is cancelled.

FR-BP-03.3	The system shall enforce class-range constraints: Economy seats are 1–30, Business seats are 31–40, First Class seats are 41–50.
FR-BP-03.4	The system shall display all currently available seat numbers in the selected class before prompting for selection.
FR-BP-03.5	Upon successful seat booking, the system shall display 'Seat selected successfully!' and record the seat number.

4.3.4 FR-BP-04: Baggage Weight and Fare Calculation

Feature ID	Feature Name	Priority	Module
FR-BP-04	Baggage Weight and Fare Calculation	High	Baggage Weight and Fare Calculation

Calculates the total fare per passenger based on destination and baggage weight.

Description	After seat selection, the user enters baggage weight (0–20 kg). The system calculates the total fare by adding a per-kg surcharge to the base destination price.
Trigger	After seat selection for each passenger.
Inputs	Baggage weight (float, 0–20 kg)
Processing	Validates weight in range [0, 20]. Fare = Base_Price + (weight × 1000). Base_Price is UAE_PRICE (100,000) for UAE or SA_PRICE (150,000) for Saudi Arabia. Accumulated into total_fare across all passengers.
Outputs	Total fare is accumulated and displayed before payment. Invalid weight: 'Weight shold be less than 20KG!'
Pre-Conditions	Seat has been selected; arrival country is known.
Post-Conditions	Passenger fare is added to total_fare.

Individual Requirements:

FR-BP-04.1	The system shall accept baggage weight as a floating-point value in the range [0, 20] kg inclusive.
FR-BP-04.2	The system shall calculate fare as: Fare = Base_Price + (Baggage_Weight × 1,000 PKR/kg).
FR-BP-04.3	The base price for UAE destinations shall be PKR 100,000 (UAE_PRICE constant).
FR-BP-04.4	The base price for Saudi Arabia destinations shall be PKR 150,000 (SA_PRICE constant).
FR-BP-04.5	The total_fare variable shall be the sum of all per-passenger fares across the booking.
FR-BP-04.6	The system shall reject baggage weights outside [0, 20] and re-prompt.

4.3.5 FR-BP-05: Booking Confirmation

Feature ID	Feature Name	Priority	Module
FR-BP-05	Booking Confirmation	High	Booking Confirmation

Presents the total bill and initiates the payment flow.

Individual Requirements:

FR-BP-05.1	Before initiating payment, the system shall display the total bill: 'Your Total Bill is = [total_fare]'.
FR-BP-05.2	The system shall allow the user to abort the booking before providing passenger details by selecting 'No' at the initial booking prompt.

FR-BP-05.3	Within the booking flow, the user must either log in to an existing account or sign up for a new account before passenger details can be entered.

4.4 Payment Module

The Payment Module processes the financial transaction for a booking. It supports multiple currencies and payment methods, validates payment sufficiency, and records the booking on success. It is implemented in the Payment class.

4.4.1 FR-PM-01: Currency Selection and Conversion

Feature ID	Feature Name	Priority	Module
FR-PM-01	Currency Selection and Conversion	High	Currency Selection and Conversion

Allows the user to pay in one of five supported currencies with automatic PKR conversion.

Description	The system presents a list of available currencies and their exchange rates to PKR. The user selects one, and all subsequent payment amount inputs are interpreted in that currency and converted to PKR before validation.
Trigger	After total fare is displayed.
Inputs	Currency choice (integer 1–5)
Processing	currency_convertor(amount, choice) multiplies the entered amount by the rate for the selected currency. Rates are stored in a map<string, float> in the Payment constructor.
Outputs	Displays: '1. Dirham : 76.37', '2. EUR : 310', '3. Pound : 360', '4. Saudi Riyal : 74.79', '5. USD : 285' Invalid: 'Invalid choice!'
Pre-Conditions	Total fare has been computed.
Post-Conditions	Selected currency is recorded; subsequent payment amounts are converted.

Supported Currencies and Exchange Rates:

Choice	Currency	Code	Rate (PKR per unit)	Example: 1 unit = PKR
1	UAE Dirham	AED	76.37	76.37
2	Euro	EUR	310.00	310.00
3	British Pound	GBP	360.00	360.00
4	Saudi Riyal	SAR	74.79	74.79
5	US Dollar	USD	285.00	285.00

Individual Requirements:

FR-PM-01.1	The system shall support payment in five currencies: UAE Dirham (AED), Euro (EUR), British Pound (GBP), Saudi Riyal (SAR), and US Dollar (USD).
FR-PM-01.2	The system shall display all available currencies with their PKR exchange rates before prompting for selection.
FR-PM-01.3	The system shall convert the entered payment amount to PKR using the formula: PKR_amount = entered_amount × exchange_rate.
FR-PM-01.4	The system shall reject currency choices outside the range 1–5 and re-prompt.
FR-PM-01.5	The fixed exchange rates stored in the system are: AED=76.37, EUR=310.00, GBP=360.00, SAR=74.79, USD=285.00 PKR per unit.

4.4.2 FR-PM-02: Payment Methods

Feature ID	Feature Name	Priority	Module
FR-PM-02	Payment Methods	High	Payment Methods

The system shall support three payment methods: Debit Card, Credit Card, and Online Banking.

Description	After currency selection, the user selects a payment method. For card payments, card details are collected. For online banking, bank account details are collected.
Trigger	After currency selection.
Inputs	Debit/Credit Card: Cardholder name, 16-digit card number, expiry date (dd/mm/yyyy), 3-digit CCV. Online Banking: Bank name, account number, receiver account number, password.
Processing	debit_card(): collects card details (applies check_valid_date on expiry). bank_account(): collects banking details. For both methods, prompts for transaction amount and calls currency_convertor + payment_checker.
Outputs	Prompts for payment details; then: 'Flight Booked Successfully' or 'Payment Insufficient' Invalid method: 'Invalid Choice!'
Pre-Conditions	Currency has been selected.
Post-Conditions	If payment succeeds, booking is recorded.

Individual Requirements:

FR-PM-02.1	The system shall support debit card payment, collecting: cardholder name, 16-digit card number, expiry date, and 3-digit CCV.
FR-PM-02.2	The system shall support credit card payment using the same input flow as debit card.
FR-PM-02.3	The system shall support online banking payment, collecting: bank name, account number, receiver account number, and account password.
FR-PM-02.4	The card expiry date shall be validated using check_valid_date() (must be a valid date in 2025–2026 range that is not in the past).
FR-PM-02.5	Card and banking details are collected for confirmation purposes only; no real payment gateway integration is performed in this version.

4.4.3 FR-PM-03: Payment Validation and Confirmation

Feature ID	Feature Name	Priority	Module
FR-PM-03	Payment Validation and Confirmation	High	Payment Validation and Confirmation

Validates that the payment amount is sufficient and confirms the booking.

Description	After payment details are entered, the system converts the transaction amount to PKR and compares it against the total fare. If sufficient, the booking is confirmed and stored. If not, the user is re-prompted.
Trigger	After payment amount is entered.
Inputs	Transaction amount (float) in selected currency.

Processing	pay_money = currency_convertor(temp_money, choice). If pay_money >= total_fare: success path. If pay_money > total_fare: displays change. If pay_money < total_fare: 'Payment Insufficient' — re-prompts.
Outputs	Exact amount: 'Flight Booked Successfully' Excess amount: 'We will send back your remaining amount soon = [change]' then 'Flight Booked Successfully' Insufficient: 'Payment Insufficient'
Pre-Conditions	Payment method and currency selected.
Post-Conditions	On success, the current Payment object is added to the static pay vector.

Individual Requirements:

FR-PM-03.1	The system shall convert the entered amount from the selected currency to PKR before comparison.
FR-PM-03.2	The system shall reject payment amounts less than the total fare and re-prompt for a new amount.
FR-PM-03.3	The system shall display the change amount if the payment exceeds the total fare: 'We will send back your remaining amount soon = [amount]'.
FR-PM-03.4	Upon successful payment, the system shall display 'Flight Booked Successfully' and append the Payment object to the static pay vector.

4.5 Receipt Module

The Receipt Module generates a printed confirmation of the completed booking. It is implemented in the final Receipt class, which inherits from Payment.

4.5.1 FR-RC-01: Receipt Generation

Feature ID	Feature Name	Priority	Module
FR-RC-01	Receipt Generation	High	Receipt Generation

Generates a complete booking receipt after successful payment.

Description	After payment is confirmed, the system generates a receipt displaying all passenger details, flight information, and the total amount paid.
Trigger	Called automatically after general_payment_process() returns (whether booking was made or declined).
Inputs	No additional user input required.
Processing	Iterates person_vector to display each passenger's details via Person::show(). Calls Search_flight::show_flights() to re-display the complete flight information. Displays the total_fare.
Outputs	'Here is your Ticket Receipt:' For each passenger: 'Passenger Details [N]:' followed by first name, last name, CNIC, passport, phone number. Flight information block. 'Total Paid amount is = [total_fare]'
Pre-Conditions	Booking flow has been completed (whether payment was made or not).
Post-Conditions	Receipt is printed to the console; no file output.

Individual Requirements:

FR-RC-01.1	The system shall generate a receipt displaying all passenger details: first name, last name, CNIC, passport number, and phone number for each passenger.
FR-RC-01.2	The system shall include the complete flight information block in the receipt.
FR-RC-01.3	The system shall display the total paid amount: 'Total Paid amount is = [total_fare]'.
FR-RC-01.4	The receipt shall be generated regardless of whether the booking was confirmed (the receipt is issued after the general payment process, even if the user declined).

4.6 Flight Cancellation Module

The Flight Cancellation Module allows a user to cancel a previously booked flight. It is implemented as the static Reciept::cancel_flight() method.

4.6.1 FR-CF-01: Cancel Flight

Feature ID	Feature Name	Priority	Module
FR-CF-01	Cancel Flight	High	Cancel Flight

Allows a user to cancel a booked flight, releasing the reserved seat.

Description	The user provides a flight ID, flight date, and their CNIC. The system searches the payment records, presents a cancellation confirmation, and if confirmed, removes the booking and frees the seat.
Trigger	User selects option 8 (Cancel Flight) from the main menu.
Inputs	Flight ID (string), Flight date (dd/mm/yyyy), CNIC (long long integer)
Processing	1. Prompt for flight ID (string). 2. Prompt for and validate flight date (2025–2026 range, valid month/day). 3. Prompt for CNIC. 4. Search the static pay vector for a record matching the CNIC. 5. If found: display cancellation confirmation prompt (1=Yes, 2=No). 6. If Yes: find the matching seat in seat_vector, call cancel_seat(), erase the payment record from pay vector, and display success. 7. If No: display 'OK As you Wish!'. 8. If CNIC not found: 'CNIC does not match!'
Outputs	Success: 'Flight cancelled Successfully!' Declined: 'OK As you Wish!' CNIC not found: 'CNIC does not match!' Seat double-cancel: 'Seat is already Freed!' Wrong seat: 'This is not your seat!'
Pre-Conditions	A booking must exist in the pay vector for the entered CNIC.
Post-Conditions	Payment record is removed from the pay vector; seat is returned to available status in the seat map.

Individual Requirements:

FR-CF-01.1	The system shall identify the booking to cancel using the user's CNIC as the primary lookup key.
FR-CF-01.2	The system shall present a cancellation confirmation prompt before proceeding with cancellation.
FR-CF-01.3	Upon confirmed cancellation, the system shall remove the payment record from the in-memory pay vector.
FR-CF-01.4	Upon confirmed cancellation, the system shall release the associated seat (set the seat back to 0 in the seat map) so it becomes available for future bookings within the session.
FR-CF-01.5	The system shall prevent cancellation of a seat that has already been freed.
FR-CF-01.6	The system shall prevent cancellation of a seat that does not belong to the requesting user.
FR-CF-01.7	The flight date entered during cancellation shall be validated to be within the 2025–2026 range with valid day and month values.

5. Non-Functional Requirements

This section defines the quality attributes and operational characteristics that the ATRS must satisfy. These requirements constrain the system's design and implementation and are directly verifiable against the source code and test execution.

5.1 Performance Requirements

NFR-PE-01	The system shall respond to all user prompts and display results within 3 seconds under normal operating conditions on the target hardware specification.
NFR-PE-02	All input validation operations (name check, date check, password check, numeric check) shall execute instantaneously (sub-millisecond) as they involve no I/O or network calls.
NFR-PE-03	The seat booking and availability check operations shall execute in O(1) time, as they access the seat array by index via a hash map.
NFR-PE-04	The user authentication search (login, check_details, edit_details) shall complete in O(n) time where n is the number of registered users in the current session.
NFR-PE-05	The total memory footprint of the application during a typical session (up to 50 bookings, 5 passengers each) shall not exceed 50 MB.

5.2 Safety Requirements

NFR-SA-01	The system shall not crash or produce undefined behaviour when the user enters non-numeric input for integer fields; it shall use cin.clear() and cin.ignore() to recover gracefully.
NFR-SA-02	The system shall not allow a booking to proceed if the destination country or airport code is invalid — it shall re-prompt until valid input is received.
NFR-SA-03	The system shall not allow a seat to be double-booked within a session; the seat state is checked before every booking operation.
NFR-SA-04	The system shall not allow flight cancellation against a CNIC that does not match any existing booking record.
NFR-SA-05	The date validation function shall prevent past dates from being accepted as valid flight departure dates.

5.3 Security Requirements

NFR-SE-01	The system shall require user authentication (login or sign-up) before any booking-related operations can be performed.
NFR-SE-02	Passwords shall be validated against a minimum complexity policy: at least 8 characters, including one uppercase, one lowercase, one digit, and one special/punctuation character.
NFR-SE-03	The forgot password recovery flow shall require verification of both email address AND phone number before allowing password reset.
NFR-SE-04	Account details (including passwords stored in the user vector) are held only in volatile memory; no credentials are written to disk or transmitted over a network in this version.
NFR-SE-05	The system shall not expose booking records of one user to another user during the same session; cancellation lookup is by CNIC and ownership is verified.
NFR-SE-06	Card details and banking credentials entered during payment are collected for user experience purposes only and are not stored beyond the immediate payment transaction.

5.4 Reliability Requirements

NFR-RE-01	The system shall handle all invalid or unexpected user inputs gracefully without terminating abnormally; all input-reading loops include error detection and re-prompting logic.
NFR-RE-02	The seat availability state shall remain consistent throughout a session; concurrent modifications are not possible in the single-threaded architecture.
NFR-RE-03	The fare calculation shall produce deterministic results: given the same destination and baggage weight, the calculated fare shall always be identical.
NFR-RE-04	The system shall correctly handle the full range of valid inputs as defined in this SRS without producing incorrect results.

5.5 Maintainability Requirements

NFR-MA-01	The system shall be implemented using object-oriented design with clear class responsibilities, enabling individual modules to be modified or extended without breaking unrelated functionality.
NFR-MA-02	Base ticket prices (UAE_PRICE, SA_PRICE) shall be defined as preprocessor macros (#define) at the top of the source file to facilitate single-point price updates.
NFR-MA-03	New destination countries or airports shall be addable by implementing new methods in the Flights class and registering them in Search_flight without modifying unrelated classes.
NFR-MA-04	Currency exchange rates shall be updatable by modifying the rates map in the Payment constructor.

5.6 Usability Requirements

NFR-US-01	All console prompts shall be clearly worded, specifying the expected input format (e.g., 'Enter Departure Date (x/y/z) = ', 'Select Departure Airport(1,2,3) = ').
NFR-US-02	All error messages shall be descriptive and actionable (e.g., 'Numbers and special characters are not allowed!', 'Weight shold be less than 20KG!').
NFR-US-03	The main menu shall list all available operations numerically, and the system shall re-display the menu after each operation completes.
NFR-US-04	Available airports and currencies shall always be displayed before the user is asked to make a selection from them.

5.7 Scalability Requirements

NFR-SC-01	The system architecture shall support the addition of new departure airports by adding new methods to the Flights class and updating the Search_flight input logic.
NFR-SC-02	The seat map (map<string, map<string, array<int,50>>>) shall correctly maintain independent seat states for each unique combination of flight ID and date.
NFR-SC-03	The passenger limit of 5 per transaction is a configurable constraint; increasing it requires modifying only the booking input validation logic.

5.8 Availability Requirements

NFR-AV-01	As a standalone console application, the system is available whenever the host machine is operational and the compiled binary is present. No server uptime or network availability is required.
NFR-AV-02	The application shall start and reach the main menu within 2 seconds of execution on the target hardware.

5.9 Backup and Recovery Requirements

NFR-BR-01	In the current version, all data is held in volatile memory. No backup mechanism is implemented; data recovery is not applicable within a session.
NFR-BR-02	A future version should implement file-based or database persistence to enable session state recovery after a crash. This is identified as a known limitation of the current implementation.

5.10 User Documentation

The following documentation components are to be delivered with the ATRS software:

- This SRS document (ATRS-2025-SRS v3.0) — serves as the authoritative requirements and design reference.
- Test Plan and Test Case Document (ATRS-2025-TP) — provides complete test coverage with 90 test cases.
- README file — provides compilation instructions (g++ -std=c++17 -o atrs main.cpp), a list of main menu options, and known limitations.
- Inline code comments — the source code contains comments explaining class responsibilities, algorithmic logic, and static member initializations.

6. Assumptions and Dependencies

The following assumptions and dependencies underpin this SRS and the ATRS implementation. If any of these assumptions prove incorrect, the corresponding requirements may need to be revised.

Assumptions:

ASSUM-01	All data entered by the user is in English. No multi-language input support is required.
ASSUM-02	The system clock of the host machine is correctly set to the local time (Pakistan Standard Time, GMT+5) and is used for date validation. Incorrect system time will affect date validation.
ASSUM-03	The compiled binary will be executed in a single-session, single-user manner. No concurrent access by multiple users is assumed.
ASSUM-04	Flight prices (UAE_PRICE = 100,000 PKR; SA_PRICE = 150,000 PKR) are fixed compile-time constants and do not change dynamically. Price updates require recompilation.
ASSUM-05	Currency exchange rates (AED=76.37, EUR=310, GBP=360, SAR=74.79, USD=285) are fixed at development time. Live rate fetching is not implemented.
ASSUM-06	The OTP code in the forgot password flow is simulated; any 4-character string is accepted. Actual OTP delivery infrastructure is not available in this version.
ASSUM-07	The maximum number of passengers per booking (5) is sufficient for the intended use cases of this academic system.
ASSUM-08	Valid flight dates are restricted to 2025 and 2026 only. This constraint is hardcoded and reflects the academic timeline of the project.
ASSUM-09	The g++ compiler (GCC 9+) or a C++17-compatible compiler is available in the target build and execution environment.

Dependencies:

DEP-01	The ATRS depends on the C++ Standard Library (STL) version compliant with ISO C++11 or later.
DEP-02	The ATRS depends on the host OS providing standard POSIX-compatible time functions (std::localtime, std::chrono::system_clock) for the date validation feature.
DEP-03	The ATRS depends on the test plan document (ATRS-2025-TP) for verification and validation activities.
DEP-04	Future versions of the ATRS will depend on a file I/O or database library (e.g., SQLite, fstream) to implement persistent data storage.

7. References

Ref. No.	Document Title	Date	Source
ATRS-2025-SRS-V1	Air Ticket Reservation System SRS v1.0	2025	Project Documentation Folder
ATRS-2025-SRS-V2	Air Ticket Reservation System SRS v2.0	2025	Project Documentation Folder
ATRS-2025-CODE	ATRS C++ Source Code	2025	Project Repository
ATRS-2025-SRS-T	SRS Template (FAST-NU)	2003	FAST-NU Academic Repository

ISO-25010	ISO/IEC 25010 – Software Quality Model	2011	ISO Standards Body
IEEE-830	IEEE Std 830-1998 – SRS Recommended Practice	1998	IEEE

8. Appendices

Appendix A – Class Hierarchy Diagram

The ATRS is implemented using the following C++ class inheritance hierarchy:

Class	Inherits From / Role
Person	Base class. Stores: first_name, last_name, phone_number, passport, cnic. Provides person_input() and show().
User_authentication	Inherits: Person. Adds email, password. Manages static user vector. Provides signup(), login(), check_details(), edit_details(), forgot_password().
Flights	Base class. Stores airport and flight data. Provides Allama_Iqbal(), faisalabad(), islamabad(), and display methods for airports.
Search_flight	Inherits: Flights. Stores search parameters. Provides input_flight_search_details() and show_flights().
Seat_management	Independent class. Manages static seat map. Provides show_*_free_seats(), book_seat(), cancel_seat().
Booking	Multiple inheritance: User_authentication + Seat_management. Manages booking flow. Provides start_Booking() and fare_calculation().
Payment	Inherits: Booking. Manages currencies and payment methods. Provides general_payment_process(), debit_card(), bank_account(), currency_convertor(), payment_checker(). Static pay vector.
Reciept (final)	Inherits: Payment. Provides generate_reciept() and static cancel_flight().

Appendix B – Complete Flight ID Reference Table

The following table lists all 36 unique flight IDs implemented in the ATRS source code (18 one-way + 18 return = 36 total):

Departure	Country	Airport	Type	Flight Name	Flight ID
Lahore	UAE	DIA	One-Way	PIA Express	LHE-DXB-30T1R
Lahore	UAE	DIA	Return	PIA Gulf1	LHE-DXB-301R
Lahore	UAE	SIA	One-Way	AirBlue Sky	LHE-SIA-20E1F
Lahore	UAE	SIA	Return	PIA Gulf2	LHE-SIA-201F
Lahore	UAE	AIA	One-Way	New PIA	LHE-AIA-30Y1R
Lahore	UAE	AIA	Return	PIA Gulf3	LHE-AIA-101A
Lahore	Saudi Arabia	KIA	One-Way	PIA Gulf11	LHE-KIA-12B09F
Lahore	Saudi Arabia	KIA	Return	PIA Gulf11	LHE-KIA-409F
Lahore	Saudi Arabia	FIA	One-Way	PIA Gulf12	LHE-FIA-A2701F
Lahore	Saudi Arabia	FIA	Return	PIA Gulf12	LHE-FIA-1C01A
Lahore	Saudi Arabia	AIA	One-Way	PIA Gulf13	LHE-AIA-BF301R
Lahore	Saudi Arabia	AIA	Return	PIA Gulf13	LHE-AIA-30S1R
Faisalabad	UAE	DIA	One-Way	PIA Express	FIA-DXB-W6301R
Faisalabad	UAE	DIA	Return	PIA Gulf1	FIA-DXB-N3501R
Faisalabad	UAE	SIA	One-Way	AirBlue Sky	FIA-SIA-S201F

Faisalabad	UAE	SIA	Return	PIA Gulf2	FIA-SIA-2041FE
Faisalabad	UAE	AIA	One-Way	New PIA	FIA-AIA-M301R
Faisalabad	UAE	AIA	Return	PIA Gulf3	FIA-AIA-1011AD
Faisalabad	Saudi Arabia	KIA	One-Way	PIA Gulf11	FIA-KIA-12L09F
Faisalabad	Saudi Arabia	KIA	Return	PIA Gulf11	FIA-KIA-2409GF
Faisalabad	Saudi Arabia	FIA	One-Way	PIA Gulf12	FIA-FIA-27P01F
Faisalabad	Saudi Arabia	FIA	Return	PIA Gulf12	FIA-FIA-320Q1F
Faisalabad	Saudi Arabia	AIA	One-Way	PIA Gulf13	FIA-AIA-F301R
Faisalabad	Saudi Arabia	AIA	Return	PIA Gulf13	FIA-AIA-V3301R
Islamabad	UAE	DIA	One-Way	PIA Express	ISB-DXB-6301R0
Islamabad	UAE	DIA	Return	PIA Gulf1	ISB-DXB-3501RZ
Islamabad	UAE	SIA	One-Way	AirBlue Sky	ISB-SIA-X201F
Islamabad	UAE	SIA	Return	PIA Gulf2	ISB-SIA-2041FQ
Islamabad	UAE	AIA	One-Way	New PIA	ISB-AIA-J301R
Islamabad	UAE	AIA	Return	PIA Gulf3	ISB-AIA-1011AO
Islamabad	Saudi Arabia	KIA	One-Way	PIA Gulf11	ISB-KIA-1209F
Islamabad	Saudi Arabia	KIA	Return	PIA Gulf11	ISB-KIA-2409F
Islamabad	Saudi Arabia	FIA	One-Way	PIA Gulf12	ISB-FIA-2701F
Islamabad	Saudi Arabia	FIA	Return	PIA Gulf12	ISB-FIA-K3201F
Islamabad	Saudi Arabia	AIA	One-Way	PIA Gulf13	ISB-AIA-F301R
Islamabad	Saudi Arabia	AIA	Return	PIA Gulf13	ISB-AIA-D3301R

Appendix C – Functional Requirements Summary

No.	Req. ID	Description	Module	Priority
1	FR-UA-01	User Sign-Up	User Auth	High
2	FR-UA-02	User Login	User Auth	High
3	FR-UA-03	View Account Details	User Auth	Medium
4	FR-UA-04	Edit Account Details	User Auth	Medium
5	FR-UA-05	Forgot Password / Password Recovery	User Auth	Medium
6	FR-FS-01	Search Available Flights	Flight Search	High
7	FR-FS-02	Flight Information Display	Flight Search	High
8	FR-BP-01	Passenger Details Entry	Booking	High
9	FR-BP-02	Flight Class Selection	Booking	High
10	FR-BP-03	Seat Selection	Booking	High
11	FR-BP-04	Baggage Weight and Fare Calculation	Booking	High

12	FR-BP-05	Booking Confirmation	Booking	High
13	FR-PM-01	Currency Selection and Conversion	Payment	High
14	FR-PM-02	Payment Methods	Payment	High
15	FR-PM-03	Payment Validation and Confirmation	Payment	High
16	FR-RC-01	Receipt Generation	Receipt	High
17	FR-CF-01	Cancel Flight	Cancellation	High